

Path of Building

C#/.NET Port Feasibility Study

Version 1.0 – 15 February 2026

*An analysis of the technical feasibility, risks, and effort required
to port the Path of Building application from Lua/LuaJIT to C#/.NET*

Contents

1. Executive Summary	3
2. Current Architecture	4
2.1. Language Breakdown	4
2.2. Application Architecture	4
2.2.1. Modifier System	4
2.2.2. Calculation Engine	4
2.2.3. Mod Parser	5
2.2.4. UI Layer	5
2.2.5. Passive Skill Tree	5
2.2.6. External Dependencies	6
3. Target Architecture: C#/NET	7
3.1. Platform and Runtime	7
3.2. Proposed Module Mapping	7
3.3. UI Framework	7
3.4. Data Migration	8
4. Technical Feasibility Assessment	9
4.1. Feasibility by Subsystem	9
4.2. Performance Considerations	9
4.3. Typing Challenges	10
5. Risk Analysis	11
5.1. Risk Register	11
5.2. The “Moving Target” Problem	11
6. Effort Estimation	13
6.1. Effort by Work Package	13
6.2. Team Size Scenarios	13
7. Migration Strategy	14
7.1. Recommended Approach: Incremental Module Port	14
7.1.1. Phase 1: Foundation (Months 1–3)	14
7.1.2. Phase 2: Calculation Engine (Months 3–6)	14
7.1.3. Phase 3: Data Model and Items (Months 4–7)	14
7.1.4. Phase 4: UI (Months 3–8, parallel with Phases 2–3)	14
7.1.5. Phase 5: Integration and Release (Months 8–10)	14
7.2. Alternative: Hybrid Approach (Embedded Lua)	14
8. Benefits of Porting	16
9. Costs and Drawbacks	17
10. Proof of Concept Recommendations	18
11. Conclusion	19

1. Executive Summary

Path of Building (PoB) is a community-maintained offline build planner for the action RPG *Path of Exile*. The application is written almost entirely in Lua, executed on LuaJIT, with a custom C/C++ rendering layer (`SimpleGraphic.dll`) backed by OpenGL ES 2.0 via ANGLE. The codebase comprises roughly **38,000 lines of core logic**, **33,000 lines of UI code**, and over **650,000 lines of game-data tables** – totalling approximately 4.2 million lines when all generated data and tree assets are included.

This study assesses the feasibility of porting the application to C#/.NET. The key findings are:

- **Technically feasible** – no fundamental blockers exist. Every subsystem has a viable .NET equivalent.
- **High effort** – conservatively estimated at **18–30 person-months** for a feature-complete port with acceptable test coverage.
- **High risk in the calculation engine** – the modifier system and damage calculations are the most complex subsystems and the hardest to port without regression.
- **Significant opportunity** – a port would unlock cross-platform deployment, stronger typing, modern tooling, and a richer UI.

The recommended approach is an **incremental, module-by-module port** with a shared test harness that validates output parity against the existing Lua implementation.

2. Current Architecture

2.1. Language Breakdown

Component	Approx. LOC	Language
Core logic (Modules/)	38,000	Lua
UI controls (Classes/)	33,000	Lua
Game data (Data/)	652,000	Lua tables
Tree data (TreeData/)	3,500,000	Lua / JSON / images
Test suite (spec/)	5,000	Lua (Busted)
Build tooling	500	Python
Native runtime	–	Pre-compiled C/C++ DLLs

Table 1: Lines of code by component

All application logic is pure Lua targeting LuaJIT 5.1. There is no C/C++ source in the repository; native functionality is consumed via pre-compiled DLLs.

2.2. Application Architecture

The application follows a custom **Model–View–Controller** pattern with an event-driven main loop:

1. **Launch.lua** – entry point, update checks, error boundary.
2. **Main.lua** – application host; loads subsystems, manages modes.
3. **Build.lua** – central build editor; coordinates tabs, calculations, and UI.
4. **BuildList.lua** – build selection / management screen.

2.2.1. Modifier System

The modifier (mod) system is the architectural heart of PoB. It models every game mechanic as a *modifier* with the following shape:

```
{ name, type, value, source, flags, keywordFlags, [tags...] }
```

Modifiers are aggregated in a ModDB (modifier database) that supports sum/more/override/flag semantics. Tags provide conditional application: Condition, Multiplier, PerStat, SkillType, etc. This system is pervasive – items, passives, skills, ascendancies, and configuration all feed into it.

2.2.2. Calculation Engine

The calculation engine is split across several files:

File	LOC	Responsibility
CalcSetup.lua	1,760	Initialise the calculation environment
CalcPerform.lua	3,597	Orchestrate the full calculation pass
CalcOffence.lua	5,873	Hit, DoT, ailment, and DPS calculations
CalcDefence.lua	3,798	Armour, evasion, block, resistances, eHP
CalcTools.lua	–	Shared helpers
CalcBreakdown.lua	–	Human-readable calculation traces

Table 2: Calculation engine modules

Key algorithms include:

- **Damage conversion chains** – physical → lightning → cold → fire → chaos, with partial conversion, added-as, and gain-as interactions.
- **Armour reduction** – $\text{armour} / (\text{armour} + \text{raw} * 5) * 100$.
- **Evasion/accuracy** – $\text{accuracy} / (\text{accuracy} + (\text{evasion} / 5)^{0.9}) * 125$.
- **Critical strike** – multi-stage calculation with lucky/unlucky variants.
- **Modifier aggregation** – efficient hash-table accumulation with conditional tag evaluation.

2.2.3. Mod Parser

ModParser.lua (6,595 LOC) converts human-readable mod text (e.g., "+25% increased Fire Damage") into structured modifier objects using hundreds of regex patterns. This is one of the most maintenance-intensive files in the project and is tightly coupled to the game's ever-changing modifier grammar.

2.2.4. UI Layer

PoB uses a **custom immediate-mode 2D GUI** built on top of SimpleGraphic.dll:

- Windowing via **GLFW**.
- Rendering via **OpenGL ES 2.0** through ANGLE (DirectX translation layer).
- 66+ control classes: buttons, dropdowns, edit fields, scroll bars, list controls, tree views, tooltips, and popups.
- Font rendering from TGA bitmap textures (Bitstream Vera Sans Mono, Fontin).
- Custom anchoring / layout system.

2.2.5. Passive Skill Tree

The passive tree subsystem handles:

- Loading and rendering tree data for 29+ game versions.
- Orbit-based node positioning with zoom and pan.
- Pathfinding for node allocation.
- Jewel socket detection and radius-based transformations.
- Timeless jewel seed-based node replacement.

Each tree version includes a tree.lua (2.7 MB), sprites.lua (627 KB), and sprite-sheet images, totalling approximately 463 MB across all versions.

2.2.6. External Dependencies

Library	Purpose
lua51.dll (LuaJIT)	Lua runtime with JIT compilation
SimpleGraphic.dll	Custom rendering engine
glfw3.dll	Cross-platform windowing
libGLESv2.dll (ANGLE)	OpenGL ES 2.0 via DirectX
libcurl.dll / lcurl.dll	HTTP networking
lzip.dll	DEFLATE compression for build codes
lua-utf8.dll	UTF-8 string handling
re2.dll + abseil_dll.dll	Google RE2 regex
zstd.dll	Zstandard compression
libwebpdecoder.dll	WebP image decoding

Table 3: Native runtime dependencies

3. Target Architecture: C#/.NET

3.1. Platform and Runtime

The recommended target is .NET 8+ (LTS), which provides:

- Cross-platform support (Windows, Linux, macOS) via a single codebase.
- Tiered JIT compilation with profile-guided optimisation (PGO).
- Native AOT compilation option for faster startup.
- `Span<T>`, `stackalloc`, and SIMD intrinsics for hot-path performance.

3.2. Proposed Module Mapping

Lua Subsystem	C# Equivalent	Notes
ModDB / ModList / Mod-Store	Strongly-typed modifier model with <code>Dictionary<string, List<Modifier>></code>	Most impactful subsystem to design well
CalcOffence / CalcDefence	Static Calculator classes or a <code>CalculationPipeline</code>	Port formulae directly; unit-test per formula
ModParser	ModParser class with compiled Regex	Use <code>GeneratedRegex</code> source generator for perf
PassiveTree / PassiveSpec	PassiveTree model + graph algorithms	Consider <code>System.Numerics</code> for vector math
Item / ItemDB	Item record types + parser	Good candidate for strong typing
Classes/ (UI)	Avalonia UI or WPF	Full rewrite; see Section 3.3
Data/ (game data)	Embedded JSON or C# source-generated models	See Section 3.4
Build XML	<code>System.Xml.Linq</code> or custom serialiser	Straightforward; maintain backwards compat
Networking / Updates	<code>HttpClient</code> + <code>System.IO.Compression</code>	Trivial

Table 4: Proposed subsystem mapping from Lua to C#

3.3. UI Framework

Three viable options exist:

Framework	Platforms	Pros	Cons
Avalonia UI	Win / Linux / macOS / Web (WASM) / Mobile	True cross-platform; active community; XAML-based; GPU-accelerated (Skia)	Smaller ecosystem than WPF; some rough edges
WPF	Windows only	Mature; rich control library; excellent tooling	Windows-only; no future cross-platform path
MAUI	Win / macOS / iOS / Android	Microsoft-backed; mobile support	No Linux; performance concerns; less mature

Table 5: UI framework comparison

Recommendation: Avalonia UI is the strongest choice. It provides cross-platform support (including Linux, a common request from the PoB community), GPU-accelerated rendering via Skia (important for the passive tree), and a familiar XAML-based programming model.

For the passive tree specifically, a custom `Avalonia.Skia` canvas control with hardware-accelerated 2D rendering would replace the current OpenGL-based approach. SkiaSharp is well-suited to this: it supports affine transforms (pan / zoom), image blitting (sprite sheets), path rendering (node connections), and text layout.

3.4. Data Migration

The game data currently encoded as Lua tables (652K LOC in `Data/`, 3.5M LOC in `TreeData/`) requires a migration strategy:

1. **Option A – Convert to JSON/MessagePack at build time.** Write a one-time Lua script that serialises all data tables to JSON. Consume in C# via `System.Text.Json` source generators for zero-reflection deserialisation. Retains the existing data pipeline (Lua export scripts) with a final conversion step.
2. **Option B – Convert to C# source-generated code.** Generate C# record types and static data from the Lua tables. Maximises type safety and IDE support but adds a code-generation step to the build.
3. **Option C – Embed a Lua interpreter.** Use `MoonSharp` or `NLua` to load Lua data tables directly. Minimises migration effort but adds a runtime dependency and complicates typing.

Recommendation: Option A for initial port (fastest path to parity), migrating to Option B over time for critical-path data (gems, base items, mod definitions) where strong typing adds the most value.

4. Technical Feasibility Assessment

4.1. Feasibility by Subsystem

Subsystem	Feasibility	Difficulty	Key Concern
Modifier system	Feasible	High	Preserving dynamic semantics in a static type system
Calculation engine	Feasible	High	Ensuring numerical parity across all edge cases
Mod parser	Feasible	Medium–High	Porting hundreds of regex patterns; ongoing maintenance
Passive tree	Feasible	Medium	Rendering performance with 1,300+ nodes and zoom
Item system	Feasible	Medium	Clipboard parsing; influence / corruption logic
UI layer	Feasible	High	Complete rewrite of 66+ controls and layout system
Data pipeline	Feasible	Medium	Automating Lua → JSON/C# conversion
Build import/export	Feasible	Low–Medium	Maintain compatibility with existing build codes
Networking	Feasible	Low	Standard HTTP; well-supported in .NET
Tree data loading	Feasible	Low–Medium	Large files; deserialisation perf matters

Table 6: Feasibility and difficulty ratings by subsystem

Overall verdict: Technically feasible. No fundamental blockers identified.

4.2. Performance Considerations

A critical question is whether .NET can match LuaJIT's performance on the calculation hot paths. Key observations:

- LuaJIT's tracing JIT is exceptionally fast for numeric code – often within 2–3x of optimised C. .NET's RyuJIT is in the same performance class.
- .NET offers advantages LuaJIT lacks: SIMD intrinsics, `Span<T>` for zero-allocation slicing, value types to reduce GC pressure, and profile-guided optimisation.
- The main bottleneck in PoB is **modifier aggregation** – iterating thousands of modifiers per calculation pass. In C#, this maps naturally to tight `for` loops over `List<T>` or arrays of structs, which RyuJIT optimises aggressively.
- LuaJIT has a **trace length limit** and a **maximum number of traces** (PoB already tunes `maxtrace=4000`, `maxmcode=8192`). .NET has no such limits.

- **Conclusion:** .NET 8+ should match or exceed LuaJIT performance for this workload, particularly with careful use of value types and avoiding unnecessary allocations on hot paths.

4.3. Typing Challenges

Lua is dynamically typed; the existing codebase relies heavily on:

- **Duck typing** – tables used as both arrays and dictionaries, sometimes simultaneously.
- **Nil propagation** – `nil` is used as a meaningful “absent” value throughout.
- **Metatables** – used for OOP (inheritance, operator overloading).
- **Dynamic dispatch** – `mod` tag evaluation dispatches on string keys to handler functions.

In C#, these patterns must be translated to:

- Explicit interfaces and abstract classes (replacing metatables).
- Nullable reference types and `Option<T>` patterns (replacing `nil` semantics).
- `Dictionary<string, Delegate>` or a visitor pattern (replacing dynamic dispatch on tag type).
- Discriminated unions via `OneOf` or a custom tagged union (for modifier value types that can be number, boolean, or table).

This is achievable but requires careful design, particularly for the modifier system where a single `mod` table can carry a wide variety of shapes.

5. Risk Analysis

5.1. Risk Register

Risk	Likelihood	Impact	Mitigation
Calculation parity regression	High	Critical	Golden-file tests comparing Lua and C# output for a large corpus of builds
UI rewrite scope creep	High	High	Time-box UI work; start with functional parity, defer UX improvements
Game patch cadence outpaces port	High	High	Automate data pipeline early; keep data layer decoupled from logic
Performance regression on calc	Medium	High	Benchmark early; profile modifier aggregation and allocation rates
Loss of contributors	Medium	High	Lua contributors may not know C#; invest in documentation and onboarding
Build code compatibility	Low	High	Implement identical Base64/DEFLATE codec; fuzz-test with real builds
Cross-platform rendering bugs	Medium	Medium	Use Avalonia/Skia (mature, tested); CI on all three OSES
Data conversion errors	Medium	Medium	Automated diffing of Lua vs JSON/C# data at build time

Table 7: Risk register

5.2. The “Moving Target” Problem

Path of Exile receives major balance patches approximately every 3–4 months. Each patch can introduce new mechanics, modify existing ones, and change hundreds of item and skill modifiers. The Lua codebase currently handles this through:

- Direct edits to `Data/` Lua tables.
- Updates to `ModParser.lua` patterns.
- New entries in `CalcOffence.lua` / `CalcDefence.lua`.

During the port, the C# codebase must be able to absorb these changes without falling behind the Lua version. This is the **single largest project-management risk**. Mitigation strategies:

1. **Maintain the Lua version** as the “source of truth” until the C# version reaches full parity.
2. **Automate data synchronisation** so that game-data updates flow into both codebases with minimal manual intervention.
3. **Port in dependency order** – data layer first, then calculation engine, then UI – so that each layer can be validated independently.

6. Effort Estimation

The following estimates assume a team of experienced C#/NET developers familiar with game mechanics, or Lua developers willing to learn C#.

6.1. Effort by Work Package

Work Package	Effort (PM)	Elapsed	Dependencies
Project setup, CI/CD, architecture	1–2	Month 1–2	None
Data pipeline (Lua → JSON/C#)	1–2	Month 1–3	Architecture
Modifier system (ModDB, ModList, ModStore)	2–3	Month 2–4	Architecture
Mod parser	2–3	Month 3–5	Modifier system
Calculation engine (offence + defence)	3–5	Month 4–7	Modifier system
Item system (parsing, DB, crafting)	2–3	Month 5–7	Mod parser, Modifier system
Passive tree (data, graph, rendering)	2–3	Month 4–7	Data pipeline, UI
UI framework + core controls	3–5	Month 3–8	Architecture
Build import/export + XML	1	Month 7–8	Item, Tree, Calc
Integration, testing, polish	2–3	Month 8–10	All
Total	18–30	10–14 months	

Table 8: Effort estimate by work package (PM = person-months)

Estimates include implementation, unit testing, and integration testing. They do not include project management overhead or community coordination.

6.2. Team Size Scenarios

Team	Elapsed Time	Notes
1 full-time developer	18–30 months	High risk of burnout and falling behind patches
2–3 full-time developers	8–14 months	Recommended; enables parallel work-streams
4–5 developers	6–10 months	Diminishing returns; coordination overhead increases

Table 9: Timeline by team size

7. Migration Strategy

7.1. Recommended Approach: Incremental Module Port

A big-bang rewrite carries unacceptable risk for a project of this size. Instead, the recommended strategy is:

7.1.1. Phase 1: Foundation (Months 1–3)

1. Set up a .NET 8 solution with an Avalonia UI shell.
2. Implement the data pipeline: Lua table → JSON → C# deserialisation.
3. Design and implement the core modifier types (Modifier, ModDB, ModList, ModStore) with comprehensive unit tests.
4. Establish a **golden-file test harness**: run a corpus of builds through both the Lua and C# calculation engines and diff the results.
5. Set up CI/CD with cross-platform builds and automated testing.

7.1.2. Phase 2: Calculation Engine (Months 3–6)

1. Port CalcSetup, CalcPerform, CalcOffence, CalcDefence.
2. Port ModParser with compiled Regex (use .NET source generators).
3. Validate every formula against the Lua implementation using the golden-file harness.
4. Target: 99.9%+ numerical parity on the build corpus.

7.1.3. Phase 3: Data Model and Items (Months 4–7)

1. Port item parsing, base items, unique items, gem data.
2. Port passive tree loading, pathfinding, and jewel logic.
3. Implement build import/export (XML + Base64/DEFLATE build codes).
4. Verify import of existing community builds.

7.1.4. Phase 4: UI (Months 3–8, parallel with Phases 2–3)

1. Implement core controls in Avalonia: list views, edit fields, dropdowns, tooltips.
2. Build the passive tree canvas with SkiaSharp (zoom, pan, node rendering).
3. Implement the build editor tabs (Skills, Items, Tree, Calcs, Config, Notes).
4. Implement the build list / management screen.

7.1.5. Phase 5: Integration and Release (Months 8–10)

1. End-to-end integration testing.
2. Community beta testing.
3. Performance profiling and optimisation.
4. Documentation for contributors.
5. Release as a parallel option alongside the Lua version.

7.2. Alternative: Hybrid Approach (Embedded Lua)

An alternative strategy embeds MoonSharp (a pure C# Lua interpreter) or NLua to run existing Lua calculation code within a C# host application. This trades reduced porting effort for:

- Runtime performance penalty (MoonSharp is 10–50x slower than LuaJIT).
- Ongoing maintenance of two languages in one codebase.

- Limited type safety and tooling benefits.

This approach could serve as a **transitional architecture** – start with a C# UI shell calling Lua calculations, then incrementally replace Lua modules with native C# as they are ported and validated.

8. Benefits of Porting

Benefit	Detail
Cross-platform	.NET 8 + Avalonia runs natively on Windows, Linux, and macOS without Wine or other compatibility layers
Type safety	Static typing catches entire classes of bugs at compile time that currently manifest as runtime nil errors in Lua
Tooling	Visual Studio / Rider provide superior debugging, profiling, refactoring, and code navigation compared to Lua editors
Performance	.NET JIT, SIMD, value types, and <code>Span<T></code> offer optimisation opportunities beyond what LuaJIT provides
Contributor pool	C# is more widely known than Lua; may attract more contributors
Modern UI	Avalonia supports data binding, styles, templates, animations, and accessibility features out of the box
Testing	xUnit/NUnit + Moq + FluentAssertions provide a richer testing ecosystem than Busted
Packaging	Single-file publish, MSIX, Snap, Flatpak, or .app bundles simplify distribution

Table 10: Benefits of a C#/.NET port

9. Costs and Drawbacks

- **Upfront investment.** 18–30 person-months of development effort before the C# version can replace the Lua version.
- **Community disruption.** Existing Lua contributors must learn C# or stop contributing to the core. This is a real risk for an open-source project.
- **Dual maintenance.** During the transition period, both codebases must track game patches, roughly doubling maintenance effort.
- **Regression risk.** The calculation engine is the project's most critical and most fragile subsystem. Any port introduces the possibility of subtle numerical discrepancies.
- **Data pipeline complexity.** A conversion step between game data export (Lua) and C# consumption adds build-system complexity.
- **Binary size.** A .NET 8 self-contained publish is larger than the current LuaJIT distribution (150–200 MB trimmed vs. 15 MB for the current runtime DLLs, excluding data).

10. Proof of Concept Recommendations

Before committing to a full port, a **time-boxed proof of concept** (4–6 weeks) should validate the highest-risk areas:

1. **Modifier system in C#.** Implement `Modifier`, `ModDB`, `ModList`, and the tag evaluation system. Verify that the dynamic dispatch and conditional evaluation semantics can be faithfully represented in C# without excessive complexity or performance overhead.
2. **CalcOffence micro-port.** Port the physical damage calculation path for a single skill (e.g., a basic attack) and compare output against the Lua version for 50+ test builds. Measure performance.
3. **Passive tree rendering.** Render the 3.26 passive tree in an Avalonia window using SkiaSharp. Implement zoom, pan, and node hover tooltips. Measure frame rate at various zoom levels.
4. **Build import.** Parse an existing PoB build code (Base64 → DEFLATE → XML) and deserialise it into C# objects. Re-export and verify round-trip fidelity.

If any of these prove unexpectedly difficult or reveal performance problems, the findings should be used to revise the effort estimates and risk assessments before proceeding.

11. Conclusion

Porting Path of Building to C#/.NET is **technically feasible** and would deliver meaningful benefits in cross-platform support, type safety, tooling, and long-term maintainability. However, the effort is substantial (18–30 person-months), the risks are real (calculation parity, community disruption, dual maintenance during transition), and the “moving target” nature of Path of Exile’s frequent patches adds ongoing pressure.

The recommended path forward is:

1. **Execute a time-boxed proof of concept** (4–6 weeks) targeting the modifier system, a subset of damage calculations, tree rendering, and build import/export.
2. **Evaluate results** against the risk register and effort estimates in this study.
3. **If the PoC succeeds**, proceed with the incremental module port strategy (Phase 1–5), targeting a 2–3 person team over 10–14 months.
4. **If the PoC reveals blockers**, consider the hybrid approach (C# UI with embedded Lua calculations) as a pragmatic alternative.

In either case, the Lua version should remain the production release until the C# version achieves full feature parity and passes comprehensive regression testing against a large corpus of community builds.